“Linux Signal-Based Process Controller with Intelligent Watchdog, Real-Time Monitoring and Fault Recovery”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

STUDENT NAME	**2520030XXX**
STUDENT NAME	**2520030XXX**
STUDENT NAME	**2520030XXX**

Under the Esteemed Guidance of

Dr. K. Anusha
Assistant Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **"Linux Signal-Based Process Controller with Intelligent Watchdog, Real-Time Monitoring and Fault Recovery"** is a record of Bonafide work of **STUDENT NAME - 2520030XXX, STUDENT NAME - 2520030XXX, STUDENT NAME - 2520030XXX** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

STUDENT NAME	**2520030XXX**
STUDENT NAME	**2520030XXX**
STUDENT NAME	**2520030XXX**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled **"Linux Signal-Based Process Controller with Intelligent Watchdog, Real-Time Monitoring and Fault Recovery"** is a bonafide work done and submitted by **Student name – 25200XXXXX ,**

in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2024-2025.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator Dr. K. Anusha, and my guide Dr. K. Anusha**,** Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the ***Department for Computer Science Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

STUDENT NAME 2520030XXX

STUDENT NAME 2520030XXX

STUDENT NAME 2520030XXX

Abstract

Operating systems manage processes, but starting a background process is far easier than supervising it correctly. A naive supervisor reports a worker as paused as soon as it has sent a signal, cannot tell a crash from a normal exit, restarts a failing program forever, confuses a slow worker with a dead one, and can signal a recycled process ID. This project, the Linux Signal-Based Process Controller with Intelligent Watchdog, Real-Time Monitoring and Fault Recovery, addresses these problems in a working system written in C and Python.

A central controller, sigctld, creates real worker processes with fork() and execv() and controls them only through POSIX signals: SIGSTOP and SIGCONT to pause and resume, SIGUSR1 and SIGUSR2 for status reports and diagnostic snapshots, and SIGTERM to stop, escalating to SIGKILL only after a grace period. Every state change is confirmed by the kernel through waitpid() and the /proc file system before it is shown. Signal handlers are async-signal-safe, workers consume signals synchronously with sigtimedwait(), and the controller runs a single-threaded poll() loop so no locks are needed.

An intelligent watchdog distinguishes an intentional stop, a normal completion, a start failure, a crash and an unresponsive worker, and recovers failures with a restart limit and exponential backoff. Clients address workers by registry ID, the controller verifies ownership in /proc before every kill(), and workers run in their own process groups and never outlive the controller. A command-line client and a Tkinter dashboard operate the system over a Unix domain socket. The project is verified by 72 unit checks, 21 integration tests against real processes and a GUI smoke test, all of which also pass under AddressSanitizer and UndefinedBehaviorSanitizer.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		8–9
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		10–11
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		12
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		13–14
		4.1 C / C++ Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	
5	System Architecture and Design		15–17
		5.1 Overall System Architecture	
		5.2System Workflow	
		5.3 Flowchart / Data Flow	
		5.4 Module Description	

		5.5 Input and Output Design	
6	**Implementation**		**18–21**
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	**Execution and Demonstration**		**22–25**
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	**Conclusion**		**26–27**
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	**References**		**28**
		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER - 1 INTRODUCTION

1.1 Background of the Project

An operating system is, at its core, a manager of processes. Every program that runs on Linux is a process created by another process, scheduled by the kernel, and finally collected by its parent. Between creation and collection, the kernel and other processes communicate with a running process through signals: small asynchronous notifications such as SIGSTOP (freeze), SIGCONT (continue), SIGTERM (please exit) and SIGKILL (exit now). Service managers, shells, container runtimes and monitoring tools are all built on these primitives.

This project, the Linux Signal-Based Process Controller, builds a small but complete supervisor on top of those primitives. A central controller process, sigctld, creates worker processes with fork() and execv(), controls them only through real POSIX signals, confirms every state change through the kernel (waitpid() and /proc), and repairs failures with a watchdog. A command-line client and a desktop dashboard let a user operate the system.

1.2 Problem Statement

Starting a background process is easy; managing it correctly is not. A naive supervisor shows a worker as paused as soon as it has called kill(pid, SIGSTOP), although the kernel may not have acted yet. It cannot tell a crash from a normal exit, restarts a broken program forever, mistakes a slow worker for a dead one, risks signalling a recycled PID that now belongs to an unrelated process, and leaves zombie processes behind. The problem addressed by this project is to design a process controller that is correct about all of these cases and can prove it with tests against real processes.

1.3 Scope of the Project

- Creating, pausing, resuming, querying and stopping real Linux worker processes using fork(), execv() and POSIX signals.
- Verified state tracking: a state change is displayed only after the kernel confirms it.
- An intelligent watchdog with restart limits and exponential backoff, and detection of unresponsive (slow but alive) workers.
- Real-time monitoring of CPU and memory through /proc, and a persistent JSON-lines event log.
- A Unix-domain-socket protocol, a C command-line client and a Python/Tkinter dashboard.
- Out of scope: distributed supervision across machines, resource limits through cgroups,

and running on non-Linux operating systems.

1.4 Objectives of the Project

1. Create multiple worker processes with fork() and execv() and manage them through a registry.
2. Control workers with SIGSTOP, SIGCONT, SIGUSR1, SIGUSR2 and SIGTERM, escalating to SIGKILL only after a grace period.
3. Confirm each action with waitpid() (WUNTRACED, WCONTINUED) and /proc instead of trusting kill().
4. Detect crashes, normal exits, start failures and unresponsive workers, and recover with bounded exponential backoff.
5. Guarantee safe ownership: never signal a process the controller does not own, and leave no zombies or orphans.
6. Provide a command-line client and a dashboard, and verify the whole system with automated unit, integration and GUI tests.

1.5 Importance of the Application

The project turns abstract operating-system topics (process creation, the process life cycle, signals, zombies, process groups, inter-process communication and the /proc file system) into one observable system. It also demonstrates professional practices that matter in real systems software: async-signal-safe handlers, defence against PID reuse, bounded input handling, privilege checks and automated testing under AddressSanitizer and UndefinedBehaviorSanitizer.

1.6 Target Users / Audience

- Students and teachers of operating systems who need a working, inspectable example of process control.
- Developers who want a reference implementation of a safe, signal-driven supervisor.
- System administrators interested in how restart policies and backoff are implemented beneath tools such as systemd.

CHAPTER - 2 OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

The project exercises the operating-system concepts listed in Table 2.1. Each concept is used in the implementation, not only described.

Concept	What it means	Where it is used
Process creation	fork() duplicates the caller; execv() replaces the copy with a new program	Controller starts every sigworker (registry.c)
Process life cycle	A process is running, stopped, or terminated until its parent reaps it	Worker state machine with confirmed transitions
Signals	Asynchronous notifications; SIGSTOP and SIGKILL cannot be caught or blocked	Pause, resume, status, snapshot, stop
Zombie processes	A terminated child stays in the process table until waitpid() collects it	waitpid() loop on every iteration; tests 09 and 14
Process groups	A set of processes that can be signalled together	setpgid() isolates each worker from terminal Ctrl-C
Inter-process communication	Pipes and sockets carry data that signals cannot	Status pipes (fd 3) and a Unix domain socket for clients
I/O multiplexing	poll() waits on many descriptors with a timeout	Single-threaded controller event loop
The /proc file system	Kernel state exposed as files	CPU, RSS, process state and parent-PID checks
Synchronization	Protecting shared data from races	Single thread, flock() single-instance lock, self-pipe

Table 2.1: Operating-system concepts and their use in the project

Two signal facts shape the design. First, standard signals are not queued: if the same signal is sent three times before it is handled, the process sees it once, so SIGCHLD is only a hint that some child changed state and the controller must loop on waitpid() until it returns zero. Second, a signal cannot carry data, so every status report travels through a pipe while the signal merely asks for it.

2.2 Existing Methods / Algorithms

Existing tool	Approach	Limitation for learning
Shell job control	kill, fg and bg on a terminal session	Interactive only; no restart policy or verification
init / systemd	Large service managers with restart policies	Complex; hides the underlying system calls
supervisord / runit	Process supervisors with restart on exit	Treat any exit similarly; limited verification of state
Kubernetes probes	Liveness checks restart unhealthy containers	Operates at container level, far above signals

Table 2.2: Existing process-management approaches

Typical supervisors restart a program whenever it exits, often with a fixed delay. Exponential

backoff, a well-known technique for reducing load during repeated failures, doubles the delay after each failed attempt up to a cap, and is used here with a limit on consecutive failures.

2.3 Proposed Approach

The proposed controller follows five rules. (1) Every action is a real signal sent to a real child, never a simulated flag. (2) Delivery is not confirmation: a request stays pending until waitpid() or the worker's reply proves it. (3) Signal handlers do the minimum (set a flag and write one byte to a self-pipe) and all real work happens in a single-threaded poll() loop. (4) The watchdog separates intentional stops, normal completion, start failures, crashes and unresponsive workers before deciding what to do. (5) Clients name workers by registry ID, never by PID, and the controller verifies ownership in /proc before each kill().

2.4 Advantages of the Proposed Approach

- Correctness: the displayed state is always the verified state.
- No locks: one thread owns the registry, so signal handlers, reaping, IPC and timers cannot race.
- Safety: PID reuse, orphan workers, zombies and unauthorised clients are all handled explicitly.
- Observability: every decision is written to a persistent JSON-lines log.
- Testability: the watchdog policy is a pure function with its own unit tests.

CHAPTER - 3 SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Component	Requirement
Processor	Any x86-64 or ARM64 CPU with two or more cores recommended
Memory	2 GB RAM minimum (4 GB recommended when running Ubuntu in WSL or a virtual machine)
Storage	200 MB free space for the project, build output and logs
Display	Any display for the Tkinter dashboard; optional for command-line use

Table 3.1: Hardware requirements

3.2 Software Requirements

Software	Version / purpose
Operating system	Linux (tested on Ubuntu 24.04, kernel 6.x)
C compiler	GCC 13 with C11 support, and GNU Make
Python	Python 3.12 with python3-tk for the dashboard
Optional	xvfb for headless GUI tests
Editor	Visual Studio Code with the WSL, C/C++ and Python extensions

Table 3.2: Software requirements

3.3 Operating System Environment - Ubuntu / WSL

The project uses Linux-only interfaces (/proc, prctl(), SO_PEERCRED and pipe2()), so it must run on Linux. On Windows it runs inside Ubuntu on WSL2: install it with wsl --install -d Ubuntu, open the project folder from the Linux file system (for example ~/linux-signal-controller) and use the Ubuntu terminal inside Visual Studio Code. On Windows 11 the Tkinter dashboard appears through WSLg. The controller refuses to run as root, so a normal user account must be used. The Makefile checks uname and stops with a clear message on a non-Linux system.

3.4 Development Tools and Compilers

The code is compiled with GCC using -std=c11 -D_GNU_SOURCE -Wall -Wextra -Wshadow -Wformat=2 and builds with zero warnings. All tests were also run with AddressSanitizer and UndefinedBehaviorSanitizer. Make automates building (make), testing (make test, make gui-test) and running (make run, make gui). The script scripts/setup_ubuntu.sh installs build-essential, python3, python3-tk, make and xvfb.

CHAPTER - 4 TECHNOLOGY AND TOOLS USED

4.1 C / C++ Programming

The controller (sigctld), the worker (sigworker) and the command-line client (sigctl) are written in C11. C gives direct access to the POSIX system-call interface and precise control over memory and signal behaviour, which is what this project studies. The code base is organised as one module per directory with one header per module in include/.

4.2 GCC Compiler

GCC compiles the three executables with strict warnings. The shared sources in src/common (utilities, JSON, configuration and /proc parsing) are linked into each program. For testing, the same sources are rebuilt with -fsanitize=address,undefined to detect memory errors and undefined behaviour.

4.3 Linux System Calls

Table 4.1 lists the principal system calls used.

System call	Use in the project
fork(), execv()	Create a worker and load the worker program
waitpid()	Reap children and learn exit, stop and continue events (WNOHANG, WUNTRACED, WCONTINUED)
kill()	Send SIGSTOP, SIGCONT, SIGUSR1, SIGUSR2, SIGTERM and SIGKILL
sigaction(), sigprocmask()	Install handlers; block signals around fork() and in workers
sigtimedwait()	Worker receives blocked signals synchronously with a timeout
setpgid(), prctl()	Own process group; PR_SET_PDEATHSIG stops orphan workers
pipe2(), dup2()	Status and command pipes between controller and worker
socket(), bind(), accept()	Unix-domain-socket server; getsockopt(SO_PEERCRED) checks the client
poll()	Wait for the self-pipe, the listening socket and worker pipes
flock()	Single-instance lock for the controller
open(), read() on /proc	CPU, memory, state and parent-PID verification

Table 4.1: Linux system calls used

4.4 POSIX Threads / Synchronization

The controller deliberately does not use POSIX threads. A single poll() loop owns the process registry, so signal handlers, child reaping, client requests and timers can never race, and no mutexes are required. Synchronization is achieved structurally: handlers only set volatile sig_atomic_t flags and write to a self-pipe; the flags are fetched and cleared with signals briefly blocked; one-line pipe messages are shorter than PIPE_BUF so write() is atomic; and a flock() lock prevents two controllers from running at once. The Python dashboard does use threads (a

background poller and action threads), but only the Tk main thread draws the interface.

4.5 Linux Commands and Utilities

- make, gcc: build and test automation.
- ps -o pid,stat,cmd and /proc/<pid>/stat: inspect process state (T means stopped).
- kill -USR1 <pid>: send a signal manually to the controller.
- ./scripts/check_zombies.sh: verify that no zombie children exist.
- xvfb-run: run the dashboard test without a display.

4.6 Other Tools / Libraries Used

- Python 3 and Tkinter: dashboard (gui/dashboard.py, dialogs.py, theme.py, client.py).
- JSON lines: persistent event log (logs/events.jsonl) and the IPC reply format.
- A small hand-written JSON writer and reader in C (src/common/json.c); no external libraries are required.
- Git and Visual Studio Code with WSL: version control and development.

CHAPTER - 5 SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The system has three layers. Clients (the dashboard and sigctl) send one request line over a Unix domain socket. The controller sigctld validates it, acts on its registry of worker processes, and replies with one JSON line. Workers are separate sigworker processes, each in its own process group, that report to the controller through a status pipe.

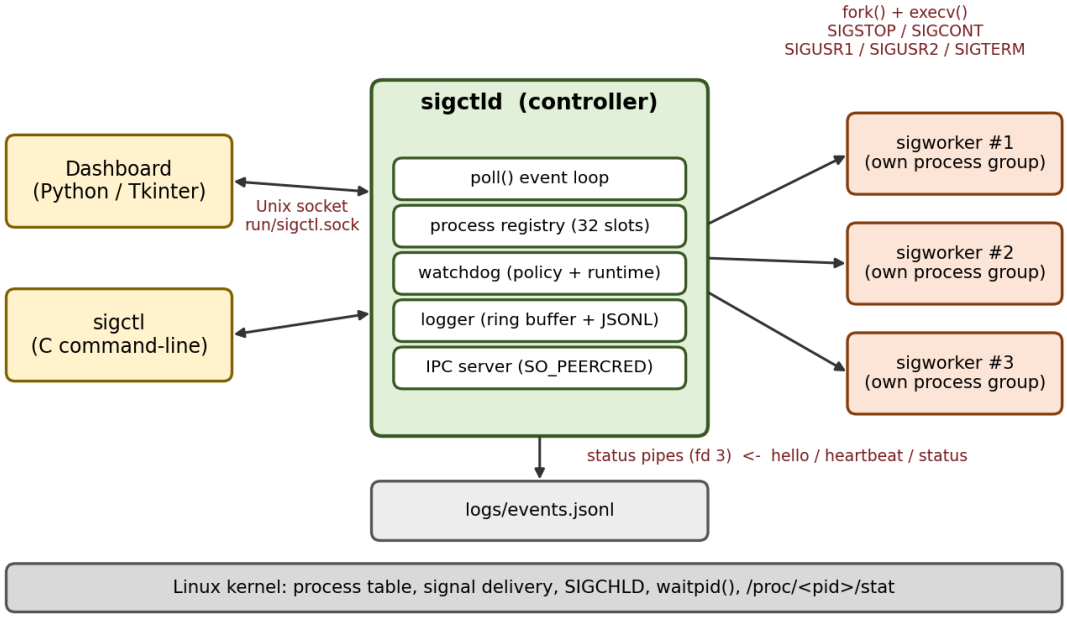


Figure 5.1: Overall system architecture

5.2 System Workflow

1. The user runs SPAWN primes through sigctl or the dashboard.
2. The IPC server checks that the client has the same user ID (SO_PEERCRED) and reads one bounded request line.
3. The command layer validates the profile and registry ID, then reg_spawn() performs fork() and execv(). The worker starts in STARTING.
4. The worker sends a hello message on its pipe; the controller marks it RUNNING.
5. For PAUSE, the controller sends SIGSTOP. The reply is delayed until waitpid(WUNTRACED) reports the stop; only then is the worker PAUSED.
6. The monitor loop reads heartbeats and /proc metrics and updates health: ok, late or unresponsive.
7. If a worker exits unexpectedly, the watchdog classifies the exit and either schedules a restart with backoff or gives up.

8. On STOP, the controller sends SIGTERM and escalates to SIGKILL only after the grace period.

5.3 Flowchart / Data Flow

Figure 5.2 shows the worker state machine. A state changes only on a confirmed event, never when a signal is merely sent. Health is tracked separately from state, because alive and responsive are different questions.

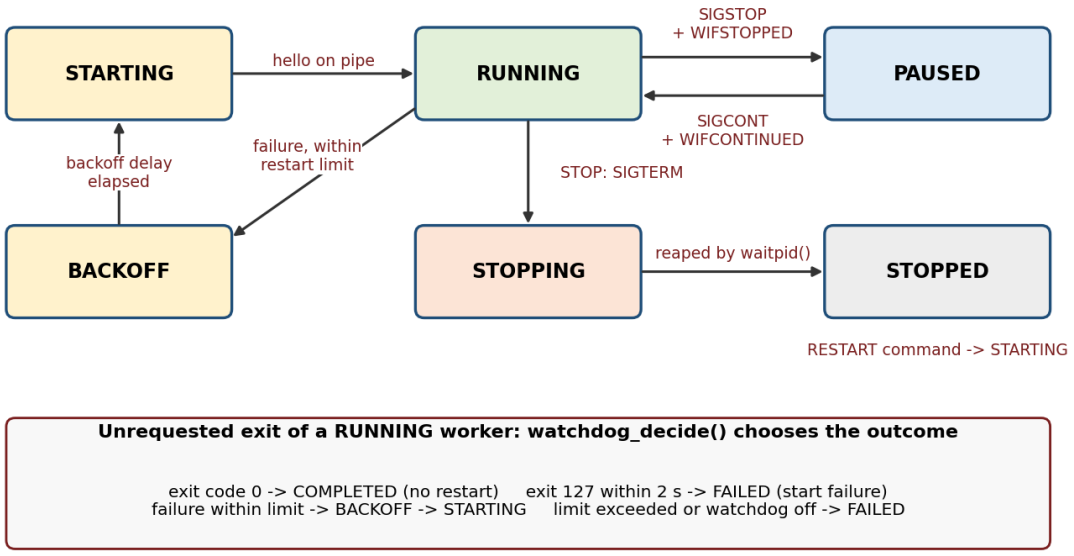


Figure 5.2: Worker state machine and watchdog outcomes

Data flows in one direction per channel: commands go from client to controller over the socket, signals go from controller to worker through kill(), worker reports return through the pipe, and every event is appended to the log.

5.4 Module Description

Module	File	Responsibility
Controller core	src/controller/main.c	Start-up checks, single-instance lock, poll() loop, shutdown, SIGHUP reload
Process registry	src/controller/registry.c	Worker slots, profiles, fork()/execv(), validated kill(), /proc metrics
Command layer	src/controller/commands.c	Parses IPC requests and tracks pending operations
Signal layer	src/signals/signals.c	sigaction() handlers, self-pipe, fork-safe masks
Watchdog policy	src/watchdog/policy.c	Pure restart/give-up decision and backoff (unit tested)
Watchdog runtime	src/watchdog/runtime.c	Reaping, confirmations, backoff timers, health checks, SIGKILL escalation
Logger	src/logger/logger.c	2000-event ring buffer and JSON-lines file

Module	File	Responsibility
IPC server	src/ipc/ipc_server.c	Socket (mode 0600), SO_PEERCRED check, bounded reads
Worker	src/worker/*.c	sigtimedwait() loop, workloads, heartbeats
CLI client	src/client/sigctl.c	Sends one command, prints the reply
Dashboard	gui/*.py	Live view and control over the socket

Table 5.1: Modules

5.5 Input and Output Design

Input is a single text line per request. Output is a single JSON line containing ok and either a message or a data object. Table 5.2 summarises the commands.

Command	Effect
PING, LIST, INFO <id>, PROFILES, CONFIG	Read-only queries
SPAWN <profile> [name]	fork() + execv() a new worker
PAUSE <id> / RESUME <id>	SIGSTOP / SIGCONT, confirmed by waitpid()
STATUS <id> / SNAPSHOT <id>	SIGUSR1 / SIGUSR2; reply returned through the worker pipe
STOP <id> / RESTART <id>	SIGTERM then SIGKILL after grace; restart starts a new process
FAULT <id> crash|segv|hang|exit	Test hook that makes a worker fail on purpose
REMOVE, STOPALL, WATCHDOG on|off, CONFIG SET/SAVE, SHUTDOWN	Management

Table 5.2: Command-line and IPC commands

Worker profiles are primes, hasher and sorter (normal workloads) and crashy, flaky and sleepy (test workers that abort after 5 s, exit with code 3 after 6 s, and answer status requests after 4 s respectively). Configuration is read from config/sigctl.conf and includes max_restarts = 3, backoff_base_ms = 1000, backoff_max_ms = 16000, stable_reset_s = 30, status_timeout_ms = 2000 and stop_grace_ms = 3000.

CHAPTER - 6 IMPLEMENTATION

6.1 Module-wise Implementation

The controller is implemented as a set of cooperating modules, all driven by the main loop in main.c. The signal layer converts asynchronous signals into flags and a pipe byte. The registry owns the table of up to 32 workers and is the only code that calls fork() and kill(). The watchdog runtime reaps children and applies the pure decisions made by policy.c. The IPC server accepts client connections without ever blocking the loop, and the logger records every event. The worker program is a separate executable so that it can be started with execv() exactly as any external program would be.

6.2 Algorithm / Pseudocode

Main loop of the controller:

```
loop:
    poll(self_pipe, listen_socket, worker_pipes..., timeout = nearest deadline)
    signals_take()                  copy-and-clear the handler flags
    reap_children()                 waitpid(-1, WNOHANG|WUNTRACED|WCONTINUED) until 0
    read worker pipes               hello / hb / status / snapshot / bye / fault
    accept clients                  one request line, one JSON reply line
    watchdog tick()                 backoff restarts, pending timeouts, health, /proc
    if shutting_down: progress_shutdown()
```

Process creation (reg_spawn):

```
access(worker path, X OK)          reject early with a clear error
pipe2(status, O_CLOEXEC); pipe2(cmd, O_CLOEXEC)
sigprocmask(SIG_BLOCK, all)        no controller handler may run in the child
pid = fork()
child:  restore default dispositions and the old signal mask
        setpgid(0, 0)                          own process group
        prctl(PR SET PDEATHSIG, SIGTERM)       die with the controller
        dup2(cmd[0], 0); dup2(status[1], 3)    pipes on stdin and fd 3
        execv(worker_path, argv)               no shell involved
parent: setpgid(pid, pid); restore mask; close child ends
        state = STARTING until the worker sends "hello"
```

Watchdog decision order: (1) a requested stop is never a failure; (2) exit code 127 shortly after start means the program could not run, so there is no restart loop; (3) exit code 0 means the task completed; (4) any other exit or death by signal is a failure, the failure count is reset if the worker had been stable for stable_reset_s, and the count is compared with max_restarts; (5) a restart is scheduled after base x 2^(n-1) milliseconds, capped at backoff_max_ms (1 s, 2 s, 4 s, 8 s, 16 s).

6.3 Core C / C++ Implementation

Signal handler of the controller (src/signals/signals.c). It only sets a flag and writes one byte, both async-signal-safe:

```
static void on signal(int sig)
{
```

```
    int saved = errno;

    switch (sig) {
    case SIGCHLD: g_chld = 1; break;
    case SIGHUP:  g_hup  = 1; break;
    case SIGUSR1: g usr1 = 1; break;
    default:      g term = 1; break;   /* SIGTERM, SIGINT */
    }
    /* Wake the poll() loop. If the pipe is full a wake-up is already
     * pending, so EAGAIN is harmless and deliberately ignored. */
    if (g_pipe[1] >= 0) {
        ssize t r = write(g pipe[1], "x", 1);
        (void)r;
    }
    errno = saved;
}
```

Watchdog policy (src/watchdog/policy.c), a pure function that is unit tested without any process:

```
int64 t watchdog backoff ms(int attempt, int base ms, int max ms)
{
    int64_t d = base_ms;
    int i;

    if (attempt < 1)
        attempt = 1;
    for (i = 1; i < attempt && d < max ms; i++)
        d *= 2;
    return d > max_ms ? max_ms : d;
}

wd_result_t watchdog_decide(const wd_input_t *in, int base_ms, int max_ms)
{
    wd_result_t r = { WD_INTENTIONAL, in->consecutive_failures, 0 };

    if (in->stop_requested) {
        r.decision = WD INTENTIONAL;
        return r;
    }
    if (in->exited && in->exit code == EXIT START FAILURE &&
        in->uptime_ms < START_FAILURE_WINDOW_MS) {
        r.decision = WD START FAILURE;
        return r;
    }
    if (in->exited && in->exit code == 0) {
        r.decision = WD_COMPLETED;
        r.failures = 0;
        return r;
    }

    /* A real failure. */
    if (in->uptime ms >= (int64 t)in->stable reset s * 1000)
        r.failures = 0;            /* it was healthy for a long time       */
    r.failures++;

    if (!in->watchdog enabled) {
        r.decision = WD DISABLED;
        return r;
    }
    if (r.failures > in->max_restarts) {
        r.decision = WD LIMIT;
        return r;
    }
    r.decision = WD_RESTART;
    r.delay_ms = watchdog_backoff_ms(r.failures, base_ms, max_ms);
    return r;
}
```

Worker side (simplified). Signals are blocked and then consumed synchronously, so printf, malloc and /proc access are safe when the worker reacts:

```
sigset t set;                          /* USR1, USR2, TERM, INT, HUP */
sigprocmask(SIG_BLOCK, &set, NULL);  /* not lost: the kernel keeps them pending */
...
sig = sigtimedwait(&set, &info, &timeout);   /* also the loop's sleep */
/* info.si_pid tells the worker WHO sent the request */
```

6.4 Linux System Calls / APIs Used

The system calls are listed in Table 4.1. Their roles in the implementation are: fork() and execv() create workers; waitpid() with WNOHANG, WUNTRACED and WCONTINUED supplies exit, stop and continue confirmations; kill() delivers signals only after /proc/<pid>/stat confirms the target is the controller's own child; sigaction() installs handlers with SA_RESTART and a full mask so handlers do not nest; sigprocmask() blocks signals around fork(); sigtimedwait() drives the worker loop; setpgid() and prctl(PR_SET_PDEATHSIG) isolate and protect workers; SIGPIPE is ignored so that writes to a closed pipe return EPIPE.

6.5 Process / Thread Implementation

Each worker is a real process in its own process group, so a Ctrl-C in the controller's terminal reaches only the controller, which then shuts down in an orderly way. Each worker calls prctl(PR_SET_PDEATHSIG, SIGTERM) and then compares getppid() with the expected parent to close the race with a parent that has already died; if the controller is killed, even with SIGKILL, all workers receive SIGTERM and exit. The controller is single-threaded. To stop a paused worker the controller sends SIGTERM followed by SIGCONT, because a stopped process cannot act on a pending SIGTERM.

6.6 IPC / Synchronization Implementation

- Client to controller: a Unix domain socket at run/sigctl.sock with mode 0600. SO_PEERCRED rejects clients of a different user. Each request is one bounded, time-limited line.
- Worker to controller: one status pipe per worker (fd 3). Messages are single JSON lines shorter than PIPE_BUF, so each write() is atomic and lines never interleave.
- Signal to main loop: a self-pipe. The handler writes one byte, which wakes poll() even if the signal arrived before poll() started.
- Single instance: flock() on the PID file; a stale socket left by a crash is detected and removed.

6.7 Memory Management Implementation

Memory use is bounded by design. The registry is a fixed array of 32 worker slots, the event ring

buffer holds 2000 events, request lines and JSON buffers have fixed maximum sizes, and no worker can grow the controller's memory. All descriptors are opened with O_CLOEXEC so they do not leak into workers. The complete test suite also passes under AddressSanitizer and UndefinedBehaviorSanitizer, which found no memory errors or undefined behaviour.

6.8 Optional User Interface Implementation

The dashboard (gui/dashboard.py) is written with Tkinter. A background thread polls the controller with LIST and EVENTS; user actions run in short action threads; only the Tk main thread draws widgets. The window shows a worker table with state, health, CPU, memory and restart counts, a details panel populated from INFO, a live event feed, and dialogs for starting workers, confirming Stop All, and editing settings. It shows only verified state and displays an offline banner when the controller stops.

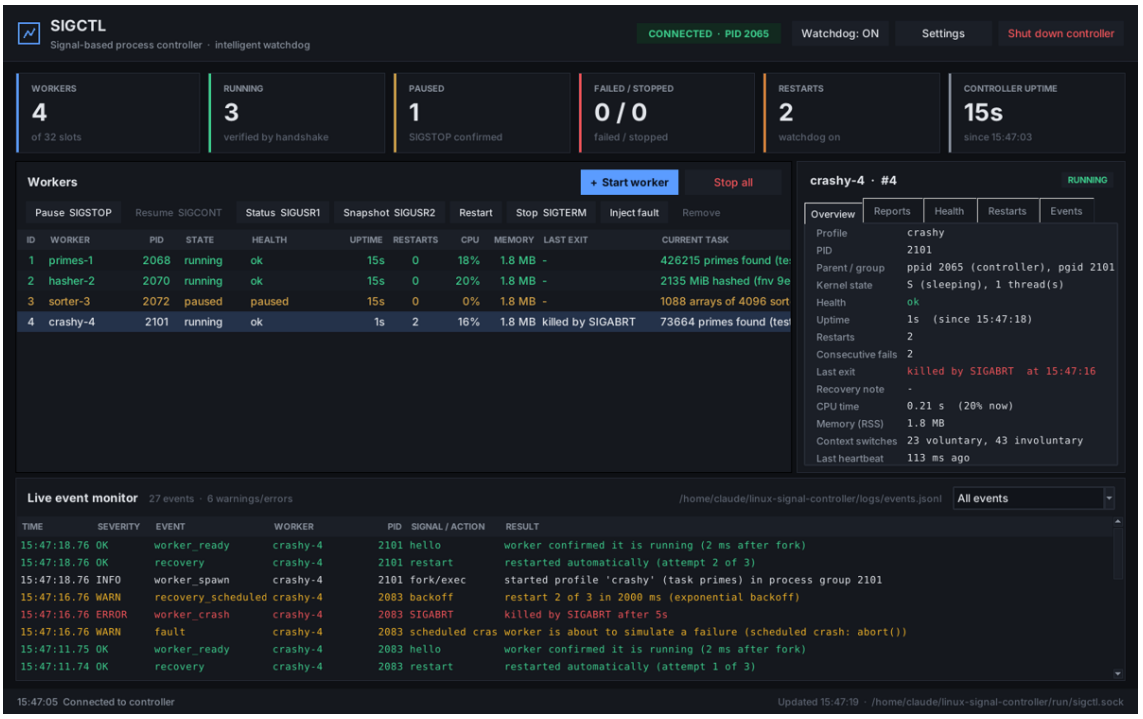


Figure 6.1: Dashboard overview with running, paused and failed workers

CHAPTER - 7 EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

```
sudo apt-get install -y build-essential python3 python3-tk   # or ./scripts/setup_ubuntu.sh
cd ~/linux-signal-controller
make          # builds build/sigctld, build/sigworker, build/sigctl
make test     # unit tests + integration tests against real processes
```

The build produces three executables with zero compiler warnings.

7.2 Execution Commands

```
# Option A: controller in background + dashboard
./scripts/start.sh

# Option B: two terminals
make run                          # terminal 1: controller, events printed live
./build/sigctl SPAWN primes       # terminal 2
./build/sigctl LIST
./build/sigctl PAUSE 1            # SIGSTOP
./build/sigctl RESUME 1           # SIGCONT
./build/sigctl STATUS 1           # SIGUSR1
./build/sigctl STOP 1             # SIGTERM
./build/sigctl SPAWN crashy       # watchdog demonstration
./build/sigctl SHUTDOWN
```

7.3 Sample Inputs and Outputs

Sample request and reply (PING and SPAWN):

```
$ ./build/sigctl PING
{"ok":true,"message":"pong from sigctld 1.0.0 (pid 213)"}
$ ./build/sigctl SPAWN primes
{"ok":true,"message":"worker started","worker":{"id":1,"name":"primes-
1","profile":"primes","pid":217,"state":"starting", ...}}
```

Excerpt of the controller's event log from a real run (workers primes-1, hasher-2, sorter-3 and crashy-4). It shows verified pause, status reply, a crash, three restarts with doubling delays, the restart limit, and a graceful stop:

```
15:47:04.499 info  worker_spawn   primes-1  pid=2068  fork/exec started profile 'primes'
15:47:04.504 ok    worker_ready   primes-1  pid=2068  worker confirmed it is running (5 ms after fork)
15:47:05.719 info  signal_sent    primes-1  pid=2068  SIGUSR1  status report requested
15:47:05.719 ok    status report  primes-1  pid=2068  reply received in 0 ms over the pipe
15:47:05.740 info  signal_sent    sorter-3  pid=2072  SIGSTOP  pause requested
15:47:05.743 ok    worker_paused  sorter-3  pid=2072  stop confirmed by waitpid(WUNTRACED) in 3 ms
15:47:10.749 error worker_crash   crashy-4  pid=2077  killed by SIGABRT after 5s
15:47:10.749 warn  recovery_scheduled crashy-4  restart 1 of 3 in 1000 ms (exponential backoff)
15:47:11.749 ok    recovery       crashy-4  pid=2083  restarted automatically (attempt 1 of 3)
15:47:23.782 warn  recovery scheduled crashy-4  restart 3 of 3 in 4000 ms (exponential backoff)
15:47:32.795 error recovery stopped crashy-4 restart limit of 3 reached; recovery stopped
15:47:47.871 ok    worker_exit    hasher-2  pid=2070  exited with code 0 after 43s (graceful stop)
```

7.4 Screenshots of Execution

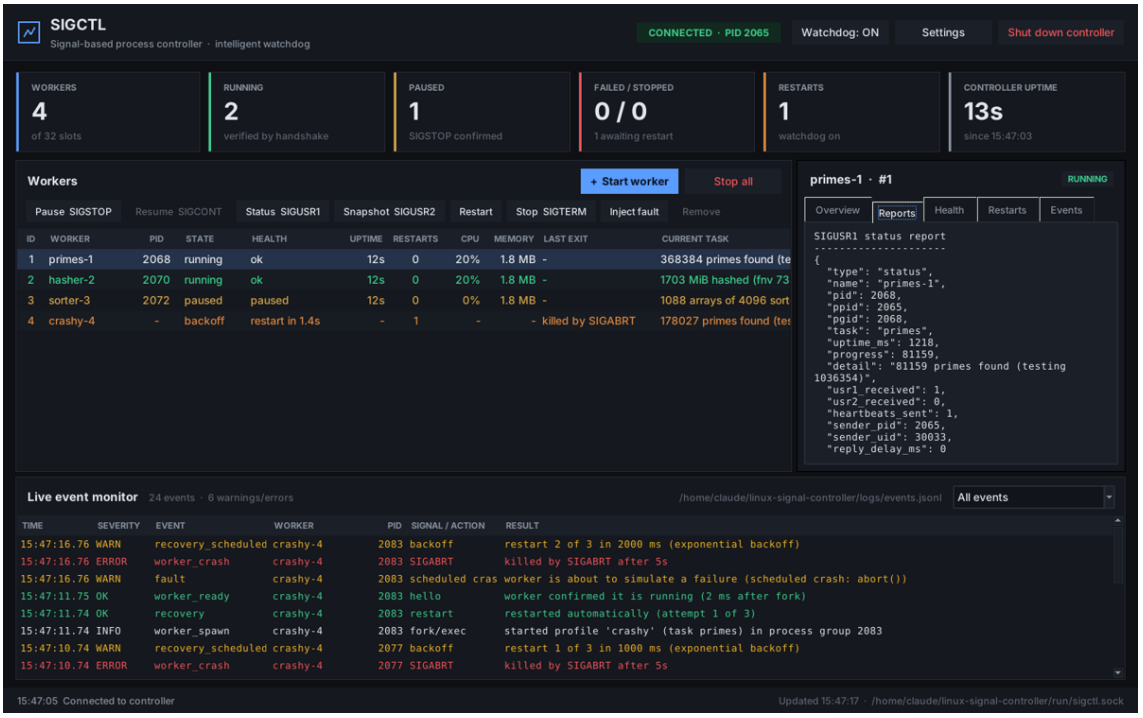


Figure 7.1: Status report returned through the worker pipe after SIGUSR1

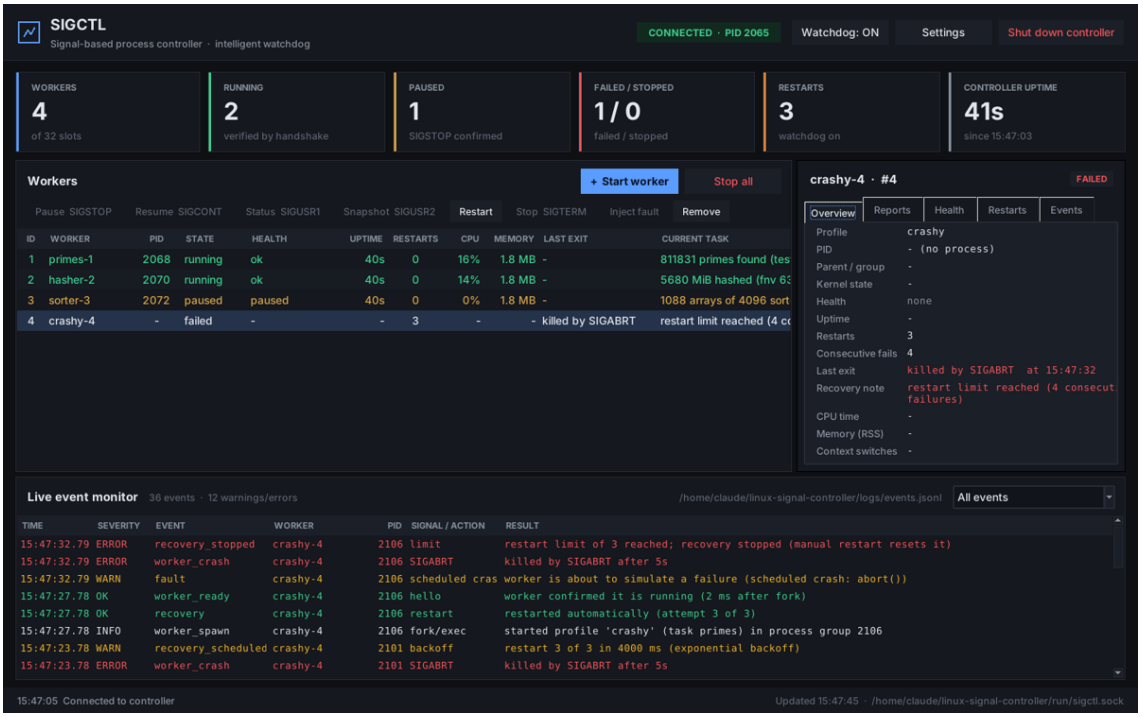


Figure 7.2: Watchdog stops restarting a crashing worker at the restart limit

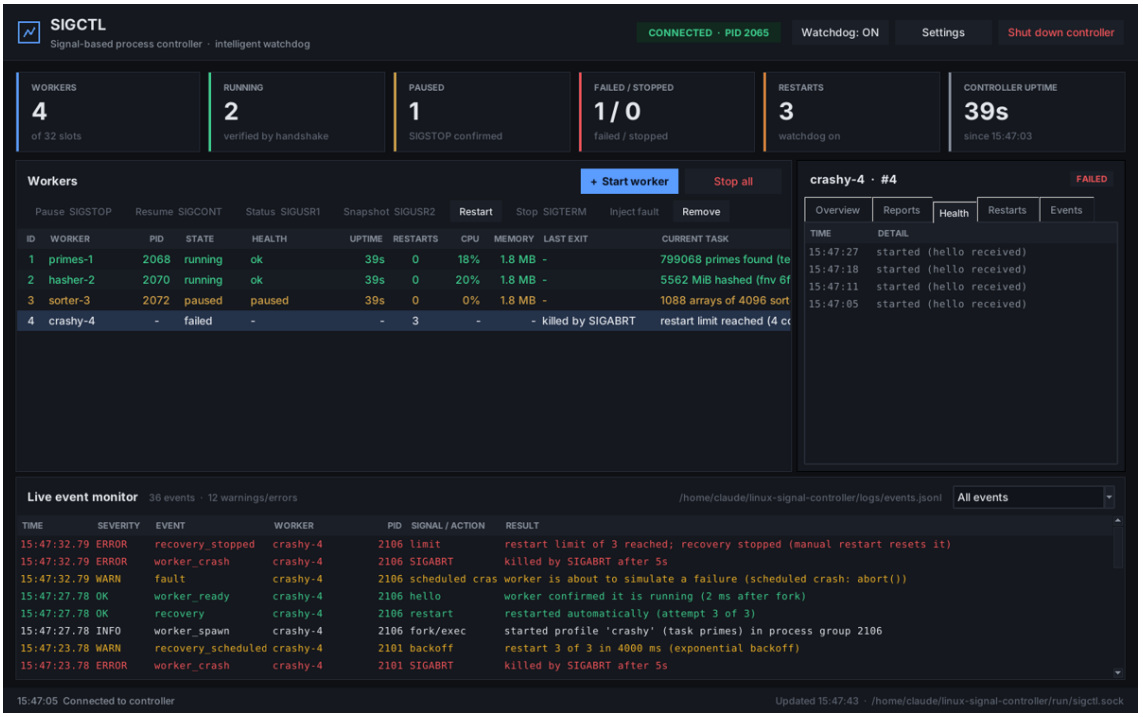


Figure 7.3: Health history of the workers

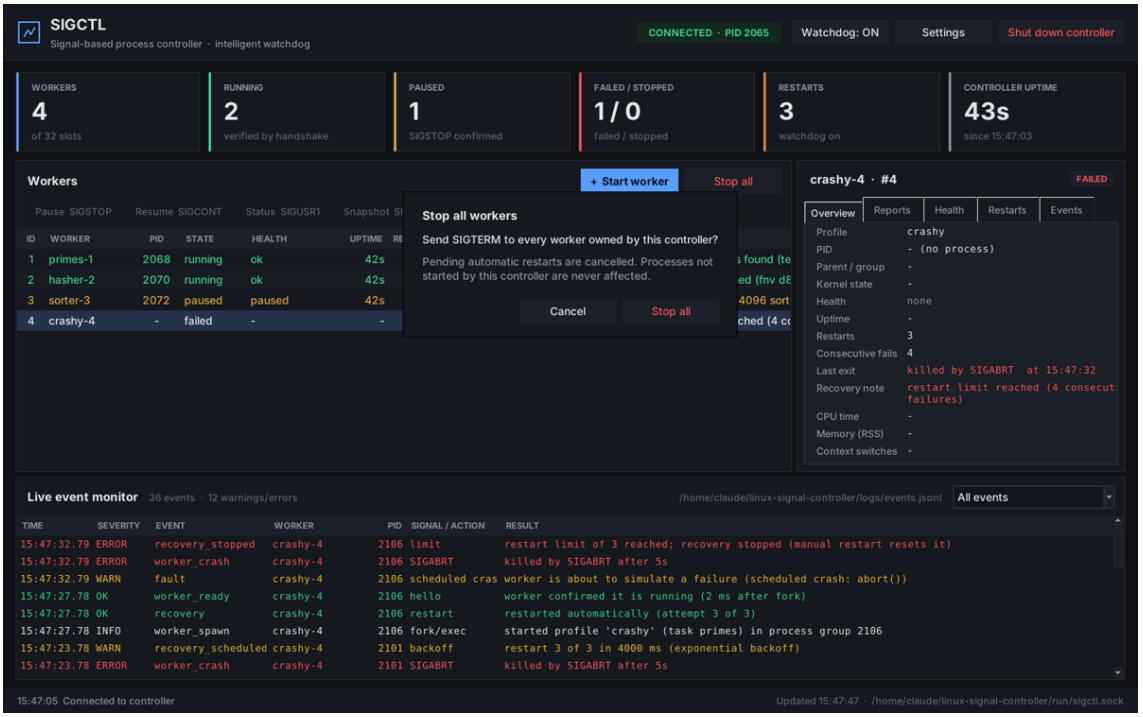


Figure 7.4: Confirmation dialog before Stop All

7.5 Demonstration of OS Concepts

Table 7.1 connects each operating-system concept to the automated test that demonstrates it. The recorded run (Ubuntu 24.04, GCC 13.3) reports 72 of 72 unit checks, 21 of 21 integration tests and 8 of 8 GUI checks passing.

Test	Scenario	Evidence

Test	Scenario	Evidence
01	Create multiple workers	3 workers, each its own process group, parent is the controller
02, 03	SIGUSR1 status, SIGUSR2 snapshot	Report received; sender_pid equals controller PID; SigBlk mask shown
04	Pause and resume	/proc state T while paused; progress frozen; running after resume
05	Graceful termination	Exit code 0, bye message, no restart
06, 07	Crash and watchdog restart	Killed by SIGABRT detected; new PID, restarts = 1
08	Restart limit	Recovery stops at the limit; manual restart recovers
09, 14	Simultaneous exits, zombies	5 exits reaped, 0 zombies; kernel children equal registry
10	Invalid IDs	Malformed IDs, raw PIDs and bad profiles rejected
11, 12, 19	IPC and log robustness	Silent or vanished clients handled; log persists; log failure reported
13	Clean shutdown	All workers (one paused) exit 0; socket removed
15	Unresponsive is not crashed	Timeout marks unresponsive; late reply returns to ok; no restart
16	Hang escalates to SIGKILL	SIGKILL only after the 1.5 s grace period
17, 18	Single instance, orphan protection	Second controller refused; workers exit if the controller is killed
20, 21	Watchdog toggle, SIGHUP reload	No restart when off; max_restarts changes after SIGHUP

Table 7.1: OS concepts demonstrated by tests

CHAPTER - 8 CONCLUSION

8.1 Summary of the Project

The Linux Signal-Based Process Controller is a working process supervisor written in C and Python. It creates real worker processes, controls them with real POSIX signals, confirms every state change through waitpid() and /proc, repairs failures with a watchdog that applies restart limits and exponential backoff, and presents verified state live in a desktop dashboard and a command-line client.

8.2 Objectives Achieved

Objective	Outcome
Create and manage workers	fork() + execv() with a 32-slot registry
Control with signals	SIGSTOP, SIGCONT, SIGUSR1, SIGUSR2, SIGTERM with SIGKILL escalation
Verified state	waitpid(WUNTRACED, WCONTINUED) and /proc confirmation
Fault recovery	Crash, exit, start failure and unresponsive cases handled; bounded backoff
Safe ownership	Registry IDs, /proc parent check, process groups, PDEATHSIG, no zombies
Interface and testing	CLI, dashboard, 72 unit checks, 21 integration tests, GUI smoke test

Table 8.1: Objectives and outcomes

8.3 Results Achieved

- All 72 unit checks, 21 integration tests (14 required scenarios and 7 extra edge cases) and 8 GUI checks passed.
- The same tests pass under AddressSanitizer and UndefinedBehaviorSanitizer.
- Five simultaneous worker exits were reaped with no zombie process.
- A crashing worker was restarted after 1 s, 2 s and 4 s and then abandoned at the limit of 3 restarts.
- A slow worker was marked unresponsive without being restarted, and recovered when it answered.

Possible future work includes resource limits through cgroups, supervision of process trees, remote management over an authenticated channel, and exporting metrics to a monitoring system.

8.4 Skills and Learning Outcomes

- Systems programming in C with POSIX process, signal and I/O interfaces.
- Writing async-signal-safe code and reasoning about signal coalescing and races.
- Designing a single-threaded event loop and a reliable restart policy.
- Building a Tkinter interface that talks to a system service.
- Automated testing against real processes and use of sanitizers.

CHAPTER - 9 REFERENCES

9.1 Books

1. A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, Wiley.
2. M. Kerrisk, The Linux Programming Interface, No Starch Press, 2010.
3. W. R. Stevens and S. A. Rago, Advanced Programming in the UNIX Environment, Addison-Wesley.
4. A. S. Tanenbaum and H. Bos, Modern Operating Systems, Pearson.

9.2 Research Papers

1. D. M. Ritchie and K. Thompson, The UNIX Time-Sharing System, Communications of the ACM, 17(7), 1974.
2. A. Baumann, J. Appavoo, O. Krieger and T. Roscoe, A fork() in the road, Proceedings of HotOS XVII, 2019.

9.3 Linux / GNU Documentation

- Linux manual pages: signal(7), sigaction(2), sigtimedwait(2), fork(2), execve(2), waitpid(2), setpgid(2), prctl(2), poll(2), pipe(2), flock(2), unix(7), proc(5).
- GNU Make Manual and GCC Online Documentation.

9.4 Online Resources

- https://man7.org/linux/man-pages/
- https://www.kernel.org/doc/html/latest/
- https://gcc.gnu.org/onlinedocs/
- https://www.gnu.org/software/make/manual/
- https://docs.python.org/3/library/tkinter.html
- https://learn.microsoft.com/windows/wsl/
- Project documentation in the docs/ folder: ARCHITECTURE.md, SIGNALS.md, PROTOCOL.md, TESTING.md.